



UNIVERSITÉ DE MONS

Département d'Informatique

Projet d'informatique – Seconde Session

Réalisé par :

Decorte Emile

222663

Encadrant : **Gauvain DEVILLEZ**

Année académique 2024–2025

Table des matières

1	Introduction	2
1.1	Contexte du projet	2
1.2	Objectif du jeu	2
2	Description	3
2.1	Description de l'application	3
2.2	Description algorithmes principaux	6
2.2.1	Logique générale	7
2.2.2	Recevoir des flottes sur une planète	7
2.2.3	Logique par rapport aux flottes	9
2.2.4	Algorithme IA dummy	10
2.2.5	Algorithme IA greedy	11
2.2.6	Algorithme IA smart	12
2.3	Tests unitaires	15
3	Points forts	16
3.1	Généralités	16
4	Points faibles	17
4.1	Smart IA	17
4.2	Utilisation de l'ia	17
5	Bugs connus	18
5.1	Envoi des flottes	18
6	Apports positifs et négatifs	19
6.0.1	Apports positifs	19
6.0.2	Apports négatifs	19

Chapitre 1

Introduction

1.1 Contexte du projet

Le projet a été introduit dans le cours de Programmation en BAB2. L'objectif principal est de réaliser un jeu reprenant les bases de SolarMax, un jeu de type conquête spatiale. Ici, le jeu est en tour par tour, intégrant une interface graphique réalisée en JavaFX.

1.2 Objectif du jeu

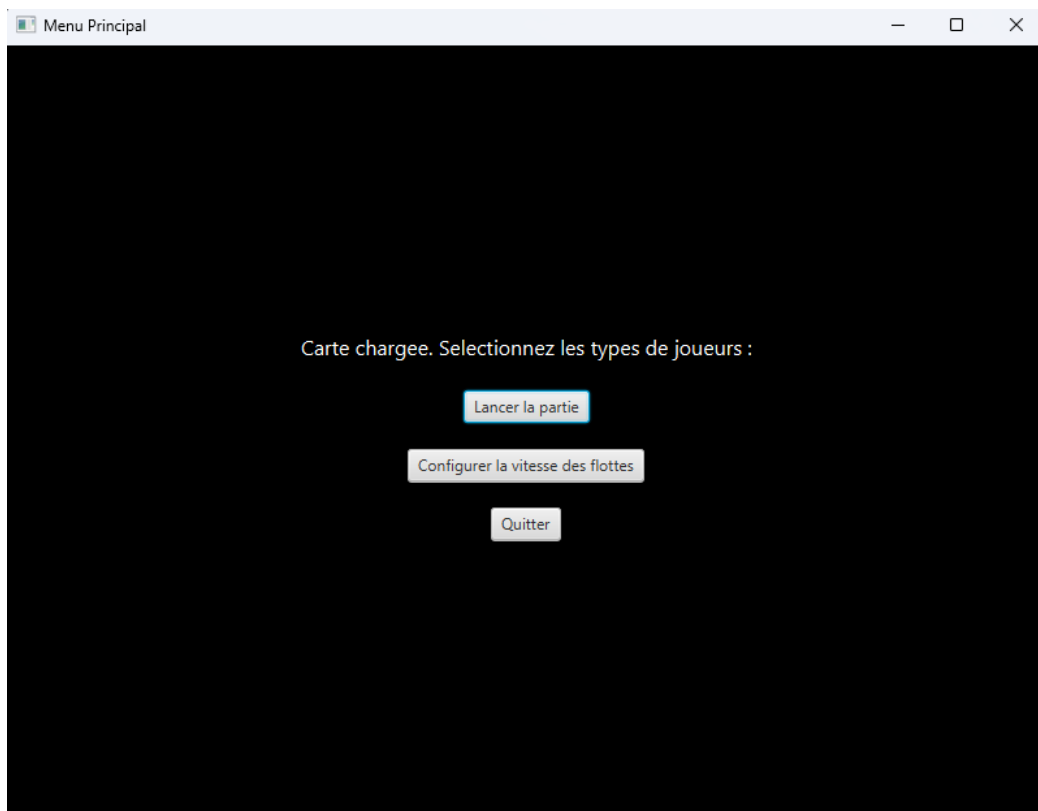
Le principe du jeu est simple : plusieurs joueurs, humains ou contrôlés par l'ordinateur, s'affrontent pour le contrôle de planètes dispersées sur une carte. Chaque planète possède une production de vaisseaux qui augmente au fil des tours. Les joueurs peuvent envoyer des flottes pour capturer des planètes neutres ou appartenant à d'autres joueurs. La partie se termine lorsqu'il n'y a plus qu'un seul joueur possédant des planètes.

Chapitre 2

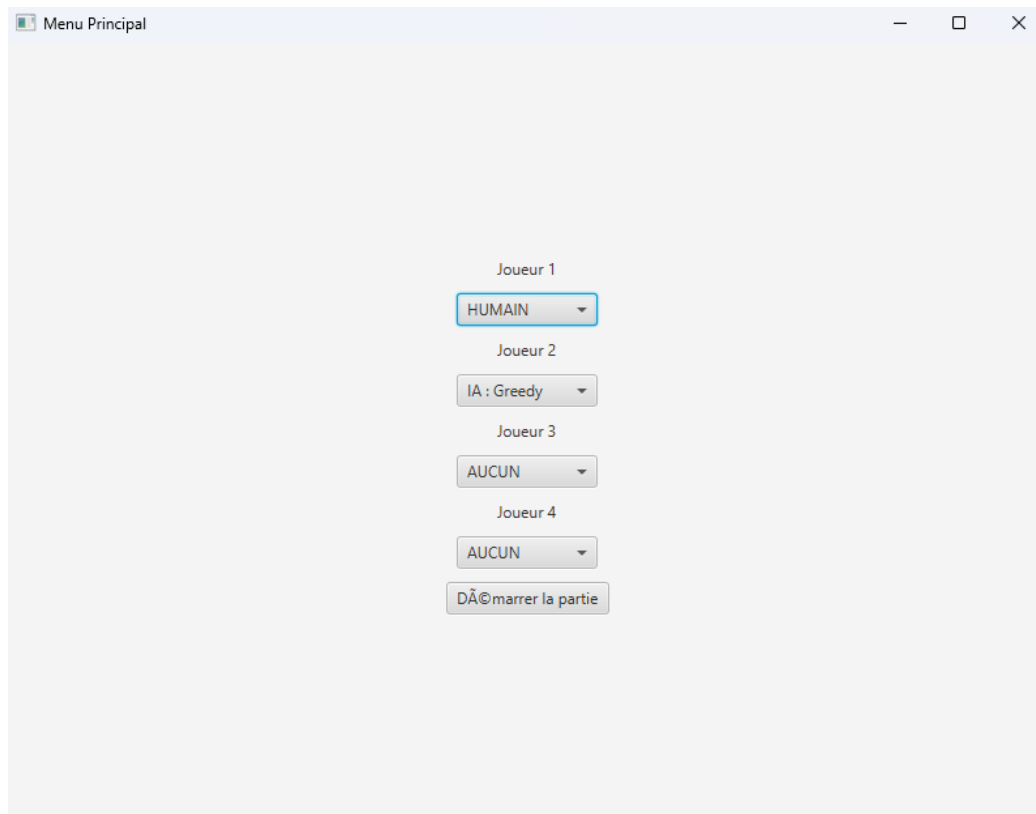
Description

2.1 Description de l'application

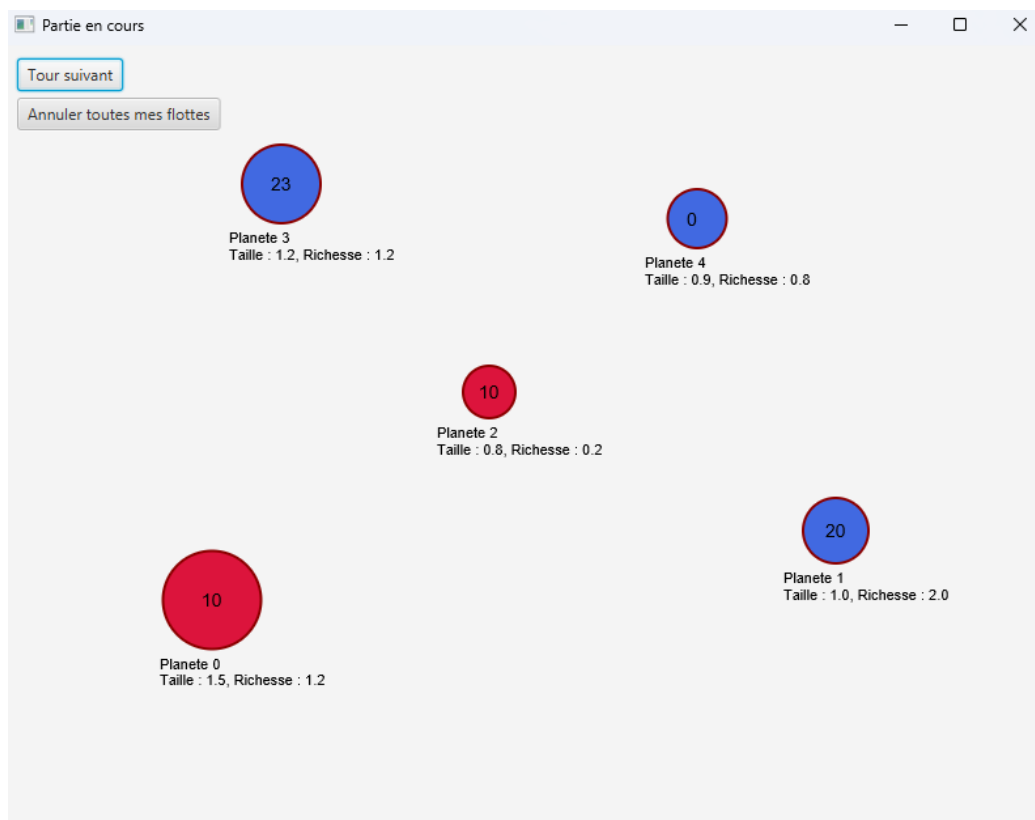
Lorsque l'on lance le jeu, on tombe sur un menu principal assez explicite. On peut soit choisir de changer la vitesse des flottes pour notre partie, de quitter le jeu. Ou bien de lancer la partie.



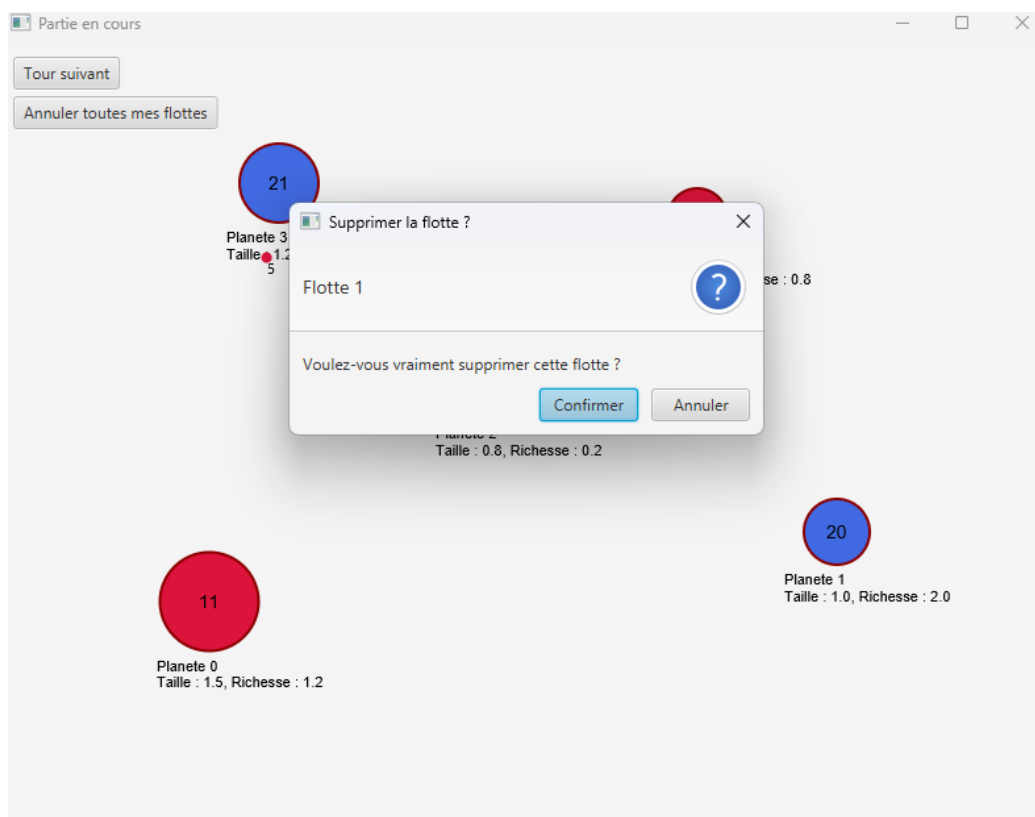
Lorsqu'on lance la partie. On a le choix de pouvoir modifier le nombre de joueurs ainsi que leurs affectations.



Les IA vont des plus simples au plus complexes :
IA Dummy = IA aléatoire, IA greedy = IA qui cherche à capturer le plus de planète possible et enfin IA smart qui part de la même base que l'IA greedy mais qui implémente un système de défense supplémentaire. Les algorithmes des IA seront décrits plus tard.



Une fois dans le jeu, les options sont simples :
On peut passer au tour suivant après avoir réalisé nos actions, On peut annuler nos flottes en cours ou bien en envoyer. Pour en envoyer il suffit de cliquer sur une de nos planètes, de sélectionner le nombre de vaisseaux à envoyer et de cliquer ensuite sur une planète ennemie ou allié. !attention voir section bug connus.

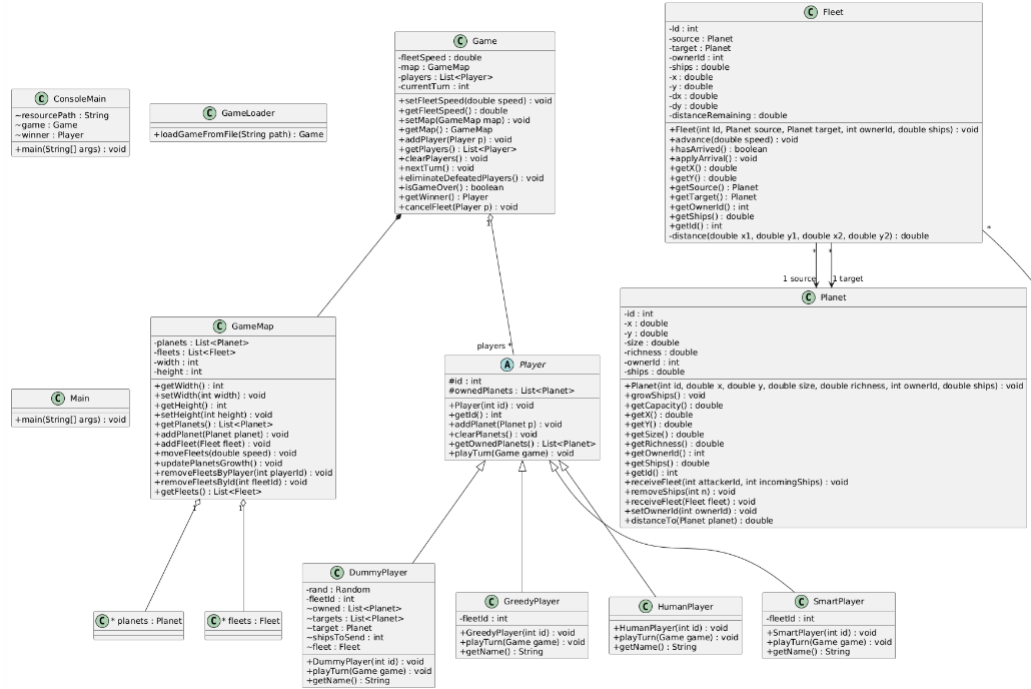


On peut aussi supprimer une flotte en particulier en cliquant dessus. Le choix de faire une suppression froide des flottes est un parti pris certes simple mais selon moi essentiel pour le jeu, en effet le seul moyen éthique de rendre les vaisseaux à un joueur aurait été de faire revenir en arrière la flotte mais cela étant dans les points supplémentaires de l'application je me suis d'abord concentré sur l'essentiel. Une fois la partie terminée, un écran de fin apparaît permettant de rejouer, de revenir au menu principal ou de quitter.

2.2 Description algorithmes principaux

Honnêtement, il n'y a pas énormément d'algorithmes complexes dans ce projet. En effet, la logique reste simple dans l'ensemble et en fonction des choix effectués pour les IA, la logique pouvait rester simple. Les quelques algorithmes un peu plus complexes sont détaillés ci-dessous

2.2.1 Logique générale



Comme mentionné plus haut, la logique est très basique. On a une partie qui possède une map et des joueurs. La map possède des planètes et des flottes. Les joueurs possèdent des Planètes. Chaque planète a un id, un id de joueur, une position, des bateaux, une richesse et une taille. Chaque joueur possède un id et des planètes. Chaque flotte est décrite par un id, une planète source et cible ainsi qu'un nombre de vaisseaux. Elle contient aussi une position, une distance restante par rapport à la cible.

2.2.2 Recevoir des flottes sur une planète

Cet algorithme reprend les lois de résolution énoncées dans l'énoncé du projet. Nous sommes dans la classe qui décrit une planète.


```

104     public void receiveFleet(int attackerId, int incomingShips) {
105         int currentShips = (int) Math.floor(this.ships);
106         if (ownerId == attackerId) {
107             this.ships += incomingShips;
108         } else {
109             int result = incomingShips - currentShips;
110             if (result > 0) {
111                 this.ships = result;
112                 this.ownerId = attackerId;
113             } else if (result < 0) {
114                 this.ships = -result;
115                 // ownerId reste inchangé
116             } else {
117                 this.ships = 0;
118                 this.ownerId = 0; // devient neutre
119             }
120         }
121     }

```

Prend en entrée l'id de l'attaquant et le nombre de vaisseaux. Si l'id attaquant est le même que celui de la planète qui reçoit on additionne les vaisseaux car ils appartiennent au même joueur. Sinon, si il y a plus de vaisseaux ennemis arrivant que de vaisseaux disponibles sur la planète, l'ennemi prend possession de la planète avec les vaisseaux ayant survécu (owner de la planète modifié). Si il y a moins de vaisseaux ennemis alors la force restante de la planète est juste réduite du nombre de vaisseaux ennemis arrivant (owner de la planète inchangé). Sinon si il y a égalité la planète devient neutre (id=0)

2.2.3 Logique par rapport aux flottes

```
public Fleet(int Id, Planet source, Planet target, int ownerId, double ships) {
    this.Id = Id;
    this.source = source;
    this.target = target;
    this.ownerId = ownerId;
    this.ships = ships;
    this.x = source.getX();
    this.y = source.getY();

    double totalDistance = distance(source.getX(), source.getY(), target.getX(), target.getY());
    this.distanceRemaining = totalDistance;

    double dirX = target.getX() - source.getX();
    double dirY = target.getY() - source.getY();

    this.dx = dirX / totalDistance;
    this.dy = dirY / totalDistance;
}
```

X et Y sont la position actuelle de la flotte, c'est utile pour la représenter en JAVAFX en direct et afin d'update lorsqu'elle bouge. dx et dy sont les vecteurs de direction vers la planète cible, ça permet de savoir dans quelle direction avancer à chaque tour. Pour ce faire on utilise dirX et dirY qui sont les vecteurs bruts, dirX est le déplacement horizontal nécessaire et Y vertical. Exemple : si la source est en (100,50) et la cible en (160,80), dirX = 160-100=60, dirY=30. Grâce à cela, et le calcul de distance total qui utilise la méthode basique de calcul de longueur entre deux points ($\sqrt{a^2 + b^2}$) . On obtient notre vecteur unitaire dx et dy. Il nous suffit de le multiplier par la vitesse pour avoir notre déplacement.

```
/**
 * Fait avancer la flotte d'une certaine distance en direction de la cible.
 *
 * @param speed la distance à parcourir ce tour-ci
 */
public void advance(double speed) {
    //éviter de dépasser
    double step = Math.min(speed, distanceRemaining);

    x += dx * step;
    y += dy * step;
    distanceRemaining -= step;
}
```

Le calcul de step permet de ne jamais dépasser la cible. Ensuite, $x += dx * \text{step}$ et $y += dy * \text{step}$ nous permettent d'avancer selon la direction dx,dy mais comme c'est unitaire on multiplie par step qui sera égal à la vitesse de base des flottes ou bien juste la distance restante par rapport à la cible si elle est plus petite. On soustrait ensuite simplement la distance parcourue à la distance restante. Exemple concret :

Flotte part de (100,50), destination (160,80).

Distance initiale = 67.08.

Vitesse speed = 10.

Tour 1 :

$\text{step} = \min(10, 67.08) = 10$

$x = 100 + 0.894 * 10 = 108.94$

$y = 50 + 0.447 * 10 = 54.47$

$\text{distanceRemaining} = 67.08 - 10 = 57.08$

2.2.4 Algorithme IA dummy

Comme dit plus haut, on se souvient qu'un joueur possède un identifiant unique ainsi qu'une liste de planètes. Pour chaque joueur, un choix de rajouter un id pour chaque flotte créée à été pris plus tard dans le projet, facilitant ainsi la suppression d'une flotte en particulier. Je pense qu'il y avait un moyen plus propre d'effectuer cela. Ici le choix a été d'initialiser une variable globale à zéro et l'incrémenter à chaque création de Flotte par joueur. Cela aurait été plus judicieux de l'implémenter directement dans le constructeur d'une Flotte mais la première idée fonctionnant j'ai fais le choix de ne plus y toucher.

```

public class DummyPlayer extends Player {
    private Random rand = new Random();
    private int fleetId;

    public DummyPlayer(int id) {
        super(id);
    }

    //Algorithme dans le rapport
    @Override
    public void playTurn(Game game) {
        List<Planet> owned = this.getOwnedPlanets();
        List<Planet> targets = game.getMap().getPlanets();

        for (Planet source : owned) {
            if (source.getShips() > 5) {
                Planet target = targets.get(rand.nextInt(targets.size()));
                if (target != source) {
                    int shipsToSend = (int) (source.getShips() * 0.5);
                    source.removeShips(shipsToSend);
                    Fleet fleet = new Fleet(fleetId, source, target, id, shipsToSend);
                    fleetId = fleetId + 1;
                    game.getMap().addFleet(fleet);
                }
            }
        }
    }
}

```

L'algorithme derrière cette ia est très simple. A chaque tour, on récupère la liste de nos planètes actualisées ainsi que les planètes générales du jeu. Ensuite, pour chacune de nos planètes ayant plus de 5 vaisseaux (Nombre arbitraire), on va choisir une planète au hasard dans les planètes globales sauf planète source (ennemies ou alliées) et envoyer la moitié de nos vaisseaux disponibles.

2.2.5 Algorithme IA greedy

N'ayant pas de consignes claires sur le comportement des ia, j'ai fait le choix d'interpréter le fait de conquérir le plus de planètes possibles au simple fait d'envoyer à chaque tour des vaisseaux sur les planètes les plus proches de chacune de nos planètes acquises. Mine de rien, cette stratégie termine souvent assez vite la partie par rapport à l'ia Dummy. Le fait d'unique-ment envoyer des vaisseaux sur des planètes ennemies répond selon moi à la consigne de "conquérir le plus de planètes possibles" car on ne se concentre pas sur la défense.

```

@Override @SuppressWarnings("unused")
public void playTurn(Game game) {
    List<Planet> owned = getOwnedPlanets();
    List<Planet> all = game.getMap().getPlanets();

    for (Planet source : owned) {

        //10 nombre arbitraire
        if (source.getShips() < 10) continue;

        Planet closest = null;
        double bestDist = Double.MAX_VALUE;

        //Planète la plus proche
        for (Planet candidate : all) {
            if (candidate.getOwnerId() != this.getId()) {
                double d = distance(source, candidate);
                if (d < bestDist) {
                    bestDist = d;
                    closest = candidate;
                }
            }
        }

        if (closest != null) {
            int shipsToSend = (int) (source.getShips() * 0.5);
            source.removeShips(shipsToSend);
            Fleet fleet = new Fleet(fleetId, source, closest, this.getId(), shipsToSend);
            fleetId = fleetId + 1;
            game.getMap().addFleet(fleet);
        }
    }
}

```

Comme pour l'ia dummy, A chaque tour on récupère nos 2 listes de planètes acquises et planètes générales (alliés et ennemies/neutre). Ensuite pour chaque Planètes acquises, si la planète à plus de 10 vaisseaux (nombre arbitraire) on va chercher la planète la plus proche. Pour ce faire, on initialise une valeur bestDist qui est égale à une distante fictive infinie. Cela nous permettra de toujours avoir au moins un résultat en dessous de ce nombre meme si il est très grand (hypothèse). Ensuite il suffit de comparer la meilleure distance actuelle (à la base +infini) à la distance actuellement calculée. Si cette distance est plus petite, on remplace la meilleure distance par cette dernière et on définit donc notre target à la planète liée a cette distance. Une fois notre target définie, on envoie la moitié de nos vaisseaux sur cette Planète.

2.2.6 Algorithme IA smart

L'objectif ici était de créer une ia "plus complexe". Cela étant assez vague, je me suis permis de répondre simplement à la question. Pour ne pas prendre de risque de prendre énormément de temps au détriment de choses plus importantes.

L'objectif ici est d'améliorer l'ia greedy en implémentant un système de défense actif. C'est à dire de répondre aux attaques ennemies en envoyant des flottes de renforts en conséquence.

```
public void playTurn(Game game) {
    List<Planet> owned = getOwnedPlanets();
    List<Planet> all = game.getMap().getPlanets();

    //Phase de défense
    defendPlanets(game, owned);

    //Phase d'attaque
    for (Planet source : owned) {

        //10 nombre arbitraire
        if (source.getShips() < 10) continue;

        Planet closest = null;
        double bestDist = Double.MAX_VALUE;

        //Planète la plus proche
        for (Planet candidate : all) {
            if (candidate.getOwnerId() != this.getId()) {
                double d = distance(source, candidate);
                if (d < bestDist) {
                    bestDist = d;
                    closest = candidate;
                }
            }
        }

        if (closest != null) {
            int shipsToSend = (int) (source.getShips() * 0.5);
            source.removeShips(shipsToSend);
            Fleet fleet = new Fleet(fleetId, source, closest, this.getId(), shipsToSend);
            fleetId = fleetId + 1;
            game.getMap().addFleet(fleet);
        }
    }
}
```

Comme vu précédemment, mis à part la phase de défense du début, le reste de l'algorithme d'attaque est le même que celui de l'ia greedy.

```
private double incomingShips(Game game, Planet p){ 1 usage  3 midectxr2
    double sum = 0;
    List<Fleet> fleets = game.getMap().getFleets();
    for (Fleet f: fleets){
        if(f.getTarget() == p && f.getOwnerId() != this.id){
            sum = sum + f.getShips();
        }
    }
    return sum;
}
```

Pour débiter la défense, on a besoin d'un algorithme qui détecte à chaque tour si des flottes sont en direction de l'une de nos planètes et en quel nombre. Pour ce faire, on récupère la liste des flotte depuis la map du jeu, on regarde pour chaque flotte si la cible est l'une de nos planètes (`f.getOwnerId()`). Si c'est le cas on incrémente notre total, sinon on ne fait rien. Une fois toutes les flottes analysées. On retourne le nombre de vaisseaux arrivant sur notre planète.

```
private void defendPlanets(Game game, List<Planet> myPlanets){ //usage Δmadeira2+1
    for(Planet p: myPlanets){
        double incoming = incomingShips(game, p);
        if(incoming > 0 && incoming>p.getShips()){
            System.out.println("danger," + incoming + "vaisseaux arrivent");

            //vaisseaux qui arrivent en aide + marge de 2
            double need = incoming - p.getShips() + 2;
            System.out.println(need);

            //on ajoute dans une liste la liste des planètes qui peuvent aider
            List<Planet> potential = new ArrayList<>();
            for (Planet planet1: myPlanets){

                //5 nombre arbitraire
                if(planet1.getShips()>need + 5 && p.getId() != planet1.getId()){
                    potential.add(planet1);
                    System.out.println(planet1.getId());
                }
            }

            //pas d'aide possible
            if(potential.isEmpty())continue;

            //si possible on cherche la plus proche
            else{
                double bestDist = Double.MAX_VALUE;
                Planet choosen = null;
                for(Planet planet2: potential){
                    double d = distance(planet2, p);
                    if (d < bestDist) {
                        bestDist = d;
                        choosen = planet2;
                    }
                }
            }
        }
    }
}
```

L'algorithme suit une logique en quatre phases : détection des menaces, calcul des besoins, identification des planètes pouvant aider, et sélection de la source optimale pour l'envoi des renforts.

On commence par parcourir l'ensemble des planètes contrôlées par le joueur afin de vérifier, pour chacune d'elles, si une menace est en approche. Pour ce faire, il calcule le nombre de vaisseaux ennemis qui se dirigent vers la planète à l'aide de la fonction `incomingShips` décrite juste avant. Si ce nombre est supérieur à zéro et dépasse le nombre de vaisseaux actuellement sur la planète, celle-ci est considérée comme en danger.

Une fois la menace détectée, l'algorithme détermine le nombre de vaisseaux supplémentaires requis pour contrer l'attaque. Ce besoin est calculé en soustrayant les forces déjà présentes sur la planète du nombre de vaisseaux ennemis attendus, puis en ajoutant une marge de deux vaisseaux.

L'étape suivante consiste à rechercher quelles planètes alliées sont candidates pour de fournir cette aide. Pour cela, on parcourt de nouveau toutes les planètes du joueur et sélectionne celles qui disposent de plus de vaisseaux que

le besoin calculé, en ajoutant encore une marge de cinq vaisseaux pour éviter qu'une planète ne se retrouve sans défense. Si aucune planète ne répond aux critères, on laisse tomber le sauvetage. Sinon, on choisit simplement la planète la plus proche dans les candidates.

Cette version possède néanmoins un gros désavantage bien que le système fonctionne. En effet, comme on regarde à chaque tour les menaces, Pour chaque nombre de tour avant que la flotte arrive à la cible on va envoyer des renforts. Cela aurait été assez simple à contrer en implémentant un cooldown si on a déjà envoyé des renforts sur cette planètes. Par exemple 2 ou 3 tours. Cela aurait empêché une bonne parties des doublons si on s'en tient à une échelle de carte similaire à l'exemple donné.

2.3 Tests unitaires

Pour cette partie, j'ai essayé de détailler au maximum directement dans le code au niveau de la documentation chaque fonction de test. Je vais m'attarder quand même sur la fonction de test par rapport à l'avancement d'une flotte.

```
@Test @midexr2 +1 *
public void testAvancementFlotte() {
    Planet source = new Planet(id: 1, x: 0, y: 0, size: 1, richness: 1, ownerid: 1, ships: 10);
    Planet target = new Planet(id: 2, x: 3, y: 4, size: 1, richness: 1, ownerid: 2, ships: 5); // distance = 5
    Fleet f = new Fleet(id: 1, source, target, ownerid: 1, ships: 5);
    f.advance( speed: 2);
    //Voir graphe dans le rapport.
    assertEquals( expected: 1.2, f.getX());
    assertEquals( expected: 1.6, f.getY());
}
```

Pourquoi c'est correct ? Source = (0,0), Target = (3,4). distance totale

$$d = \sqrt{(3-0)^2 + (4-0)^2} = \sqrt{9+16} = \sqrt{25} = 5$$

Le vecteur unitaire est donnée par :

$$\vec{u} = \left(\frac{3-0}{d}, \frac{4-0}{d} \right) = \left(\frac{3}{5}, \frac{4}{5} \right) = (0.6, 0.8)$$

En multipliant cela par 2 qui est la vitesse des flottes (arbitraire), on obtient bien qu'après un tour, on arrive à

$$(0 + 1.2, 0 + 1.6) = (1.2, 1.6)$$

Chapitre 3

Points forts

3.1 Généralités

La plus grande force de ce projet est en meme temps sa plus grande faiblesse. Sa simplicité. En effet, lors de la réalisation de ce projet, j'ai choisi tout les choix les plus simples afin de répondre à la consigne. Cela m'a permis de pouvoir avoir quelque chose de clair et bien structuré. Il y a peu de redondance dans les fonctions, le but était d'essayer avec mes connaissances actuelles de faire un code assez propre et lisible plutot que de me perdre dans de la personnalisation. A chaque fois que je créais une fonction je regardais si une autre ne me permettait pas déjà de récupérer le meme output. De plus, un des points fort de ce projet est que, sauf si erreur de ma part, je coche toute les cases demandées, que ce soit le chargement de la map, les 3 ia, la selection des joueurs avant de lancer la partie, les annulations de flottes et leur vitesse. Pour terminer, je dirais que l'ergonomie est plutot correcte, les boutons fonctionnent, il y a peu de bug lors des parties. Ps : utilisation de LaTeX à la place de Gdoc.

Chapitre 4

Points faibles

4.1 Smart IA

Cette IA est tout sauf smart, elle est très basique et n'apporte pas grand chose au projet. C'est la première chose que j'améliorerais si j'en avais l'occasion.

4.2 Utilisation de l'ia

Lors de mon projet, j'ai à plusieurs reprises eu recours à chatGPT. Cela m'a été très utile lors de la réalisation de la javadoc qui est un travail assez redondant et basique, d'où l'utilisation de l'ia. Malgré cela, je sais expliquer chacun de mes algorithmes. Je l'ai aussi utilisé lors de la réalisation de l'algorithme de lecture de fichier. En effet, l'idée d'utiliser des tokens ne vient pas de moi. Malgré cela, je me suis moi même renseigné sur comment ils fonctionnaient afin de les implémenter avec l'aide de chatGPT. A part cela, je n'y ai eu recours que pour m'aider à résoudre certains bugs mineurs ou certains bouts de code en JavaFX. Je pense notamment à l'idée d'utiliser un Group pour représenter les flottes ou la fenêtre de fin de partie.

Chapitre 5

Bugs connus

5.1 Envoi des flottes

Ce point est crucial car il peut nuire au déroulement de la partie si il n'est pas connu. Après avoir cliqué sur ANNULER dans la fenêtre de confirmation d'envoi de flotte depuis une de nos planètes, on ne peut plus directement recliquer sur une de nos planètes pour envoyer des flottes. Pour continuer à jouer, il faut cliquer sur une planète ennemie. Cela débloque la situation. De plus, N'ayant pas implémenter le fait de devoir absolument cliquer sur une planète après avoir confirmé le nombre de vaisseaux à envoyer, on peut continuer à jouer sans choisir alors qu'on a une flotte à envoyer en buffer.

Chapitre 6

Apports positifs et négatifs

6.0.1 Apports positifs

Tout d'abord, c'est le premier projet où j'utilise LaTeX pour effectuer le rapport. Cela m'a permis de développer beaucoup de connaissance de ce "language" ce qui me sera très utile pour la suite. Etant un peu plus expérimenté que lors de mon premier projet, j'ai pu expérimenter beaucoup plus le débogage. En effet, avant j'avais tendance à vite demander de l'aide aux gens plus expérimentés pour m'aider car j'étais totalement perdu. Maintenant j'ai appris comment faire pour trouver les bugs dans mon code seul. Ce projet m'a aussi permis d'approfondir mes connaissances en java et javaFX. En effet, étant mon second projet de ce genre, j'ai pu utiliser des fonctions ou des objets que je ne connaissais pas auparavant car trop complexes.

6.0.2 Apports négatifs

Le gros problème avec ce projet pour ma part est la redondance. N'ayant pas réalisé le projet de GL lors de ma première BAB2, je me retrouve à devoir refaire un projet de type jeu local en javaFX comme lors de ma BAB1. De plus ayant eu 13,5/20 la première fois, j'ai eu du mal à comprendre le fait que je devais repasser ce genre de cours. Bien que je comprenne que si je ne le passe pas cette année il y aura un problème de crédits. En conclusion, le projet étant très redondant à celui que j'ai effectué l'année dernière (jeu de type bataille navale) j'ai eu du mal à trouver la force de mettre mon coeur à ce projet. D'où la simplicité de ce dernier.